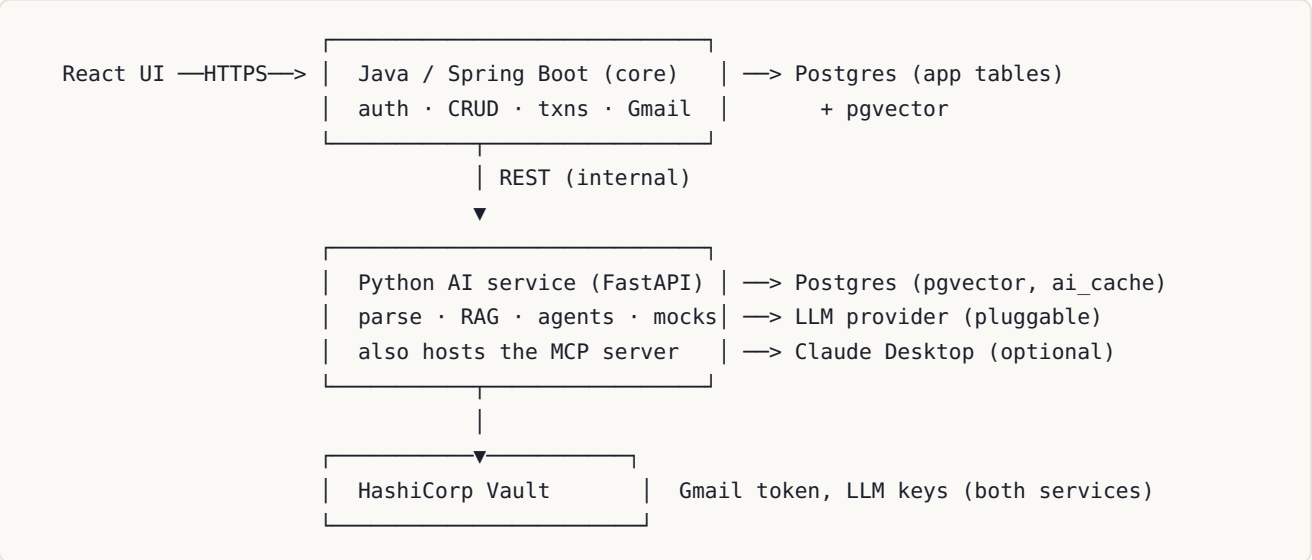
Interview Prep Platform — Backend Development Specification

Version 2.0 · Consolidated, development-ready · Single-user, remote-capable

This is the complete backend build document. It supersedes v1.0 and Addendum A and folds in every decision made: two services (Java monolith + Python AI service), Postgres + pgvector, HashiCorp Vault for secrets, single-user auth, LLM-agnostic provider, budget metering with cheap-tier fallback, adaptive plan, resume-driven onboarding, and the full feature set (mock interviewer, readiness score, spaced repetition, company prep packs, STAR bank, gap insight, Gmail job feed, interview tracker, notes, code viewer).

---

1. Architecture



Responsibility split - **Java core** = system of record. Auth, all relational CRUD, transactions, the deterministic feedback-loop rule, Gmail ingestion, budget enforcement gateway. - **Python AI service** = the brains. Resume parsing, plan generation, embeddings + RAG, all agents (job-fit, debrief, coach, mock interviewer, prep-pack), provider abstraction, RAGAS eval. Also exposes the same tools as an MCP server. - **Agent tools call back into Java REST** for app data; Python directly owns only pgvector + `ai_cache`. Clean data boundary: Java owns relational tables, Python owns vectors/cache.

Why two services: the GenAI ecosystem (RAGAS, LangGraph, embeddings) is Python-native, and the AI layer has a clean boundary. This is a deliberate, justified split — not microservice sprawl. Everything runs locally via one `docker-compose` (Postgres, Vault, Java, Python).

---

2. Tech stack

Concern	Choice
Core service	Java 21, Spring Boot 3.3+ (Web, Data JPA, Security, Validation)
AI service	Python 3.11+, FastAPI, LangGraph (agents), RAGAS (eval)
DB	Postgres 16 + pgvector extension
Migrations	Flyway (Java side owns schema)
Secrets	HashiCorp Vault (Docker), Spring Cloud Vault + hvac (Python)
LLM	Pluggable provider; default Anthropic (Claude); Bedrock/OpenAI/local supported
Auth	Spring Security, single user, JWT bearer tokens
File storage	`FileStore` interface: local disk (dev) → S3 (remote)
Packaging	docker-compose: postgres, vault, core, ai

3. Cross-cutting concerns

3.1 Auth (single user, remote-capable)

- One `app_user` row. Login (`POST /api/auth/login`) returns a JWT; all `/api/**` except auth require `Authorization: Bearer`.
- Every table carries `user_id` (FK) for isolation, even though there's one user now — makes remote hosting and any future multi-user safe.
- Java↔Python internal calls use a shared service token (from Vault), not user JWTs.

3.2 Secrets (Vault)

- Vault stores: Gmail OAuth refresh token, LLM API key(s), DB credentials, the inter-service token. **No secret in plaintext config or DB.**
- Java: Spring Cloud Vault loads secrets at startup. Python: `hvac` client reads on boot.
- DB stores only *references* (e.g. `llm_provider="anthropic"`), never keys.
- Dev bootstrap: Vault in dev mode via Docker; a seed script writes initial secrets.

3.3 Budget metering + cheap-tier fallback

- Onboarding sets `monthly_budget_usd`. Every LLM call writes a `usage_log` row (tokens in/out, model, estimated cost).
- Before any frontier-model call, `BudgetGuard` checks spend-this-month vs cap:
- under cap → proceed on requested tier.
- **over cap → force cheap tier + set a `warning` flag in the response** (never hard-block).
- `/api/usage` exposes current spend + remaining for the in-app meter.

3.4 Error & response model

- `{ "error": { "code", "message" } }`, HTTP 400/401/404/409/422/500.
- AI responses include `{ "meta": { "model", "cached", "budgetWarning" } }`.

4. Data model (complete)

Postgres. All tables: `bigint generated always as identity` PK, `user_id bigint FK`, `created_at timestamptz default now()`, `updated_at` where edited. Flyway owns DDL.

4.1 User, settings, budget

app_user — `id, email (unique), password_hash, display_name, created_at`

user_settings (onboarding output; one row per user) | column | type | notes | |---|---|---| | id | bigint PK | | | user_id | bigint FK | | | target_role | varchar(160) | "Staff backend engineer" | | target_level | varchar(40) | junior/mid/senior/staff/principal | | prep_weeks | int | duration of preparation | | hours_per_week | int | drives pace | | interests | jsonb | extra tracks, e.g. ["agentic-ai"] | | monthly_budget_usd | numeric(8,2) | LLM budget cap | | llm_provider | varchar(40) | "anthropic" | "bedrock" | "openai" | "local" | | llm_model_strong | varchar(60) | frontier model id | | llm_model_cheap | varchar(60) | cheap-tier model id |

usage_log (one row per LLM call; the budget ledger) | column | type | notes | |---|---|---| | id | bigint PK | | | user_id | bigint FK | | | ts | timestamptz | | | source | varchar(60) | agent/endpoint name | | model | varchar(60) | | | tokens_in | int | | | tokens_out | int | | | cost_usd | numeric(10,4) | estimated |

4.2 Resume profile

resume_profile (parsed once at onboarding, user-confirmed) | column | type | notes | |---|---|---| | id | bigint PK | | | user_id | bigint FK | | | raw_text | text | extracted resume text | | file_ref | varchar(500) | stored original (FileStore) | | skills | jsonb | [{name, level, years}] | | domains | jsonb | ["payments","streaming"] | | seniority | varchar(40) | inferred | | total_years | numeric(4,1) | | | experiences | jsonb | [{company, role, dates, highlights[]}] for RAG chunking | | gaps | jsonb | inferred weak areas vs target role | | confirmed | boolean | user edited & accepted |

resume_chunk (for pgvector RAG; written by Python) | column | type | notes | |---|---|---| | id | bigint PK | | | user_id | bigint FK | | | text | text | one experience/skill chunk | | embedding | vector(1024) | pgvector; dim per embedding model |

4.3 Study plan (user-scoped, adaptive)

phase — `id, user_id, code, name, icon, blurb, display_order` **week** — `id, user_id, phase_id FK, code, title, display_order`

topic | column | type | notes | |---|---|---| | id | bigint PK | | | user_id | bigint FK | | | week_id | bigint FK | | | code | varchar(40) | display code | | slug | varchar(80) | stable id (unique per user) | | title | varchar(200) | | | tag | varchar(16) | `new` | `refresh` | `dsa` | `exp` | | source | varchar(16) | `resume` | `standard` | `interest` | `custom` | | concept | text | deep-dive | | points | jsonb | deep-dive bullets | | angle | text | interview angle | | status | varchar(12) | `todo` | `done` | | is_custom | boolean | | | confidence | int | 0–100, decays (spaced repetition) | | last_reviewed_at | timestamptz | | | display_order | int | |

> `source` lets the UI show why a topic exists (resume-derived vs standard track vs your interest like agentic-ai). `confidence` + `last_reviewed_at` drive spaced repetition and the readiness score.

4.4 Topic content

note — `id, user_id, topic_id FK (unique), content_md, updated_at` **attachment** — `id, user_id, note_id FK, file_ref, original_name, mime_type, size_bytes` **resource** — `id, user_id, topic_id FK, label, url, display_order` **exercise** — `id, user_id, topic_id FK, title, repo_url, done, display_order` **question** — `id, user_id, topic_id FK, text, display_order` (study-side Q-bank)

4.5 Interviews & feedback loop

interview | column | type | notes | |---|---|---| | id | bigint PK | user_id FK | | company | varchar(160) | | | role | varchar(200) | | | stage | varchar(16) | `applied` | `screening` | `onsite` | `offer` | `rejected` | | round | varchar(120) | | | scheduled_at | date | | | outcome | varchar(16) | `pending` | `debrief` | `passed` | `offer` | `rejected` | | jd_text | text | job description (pasted at log time if not from a job_alert) | | job_alert_id | bigint FK null | link if it came from the feed | | notes | text | |

interview_question (the loop) | column | type | notes | |---|---|---| | id | bigint PK | user_id FK | | interview_id | bigint FK | | | topic_id | bigint FK null | tagged topic | | text | text | | | self_rating | int | 1–5 |

> `self_rating <= 2` → Java creates a `review_flag(source=interview)` and lowers `topic.confidence`. Guaranteed in code; the debrief agent adds tagging/scheduling on top.

review_flag — `id, user_id, topic_id FK, source(interview|spaced), reason, resolved, created_at`

4.6 Prep packs, STAR, mocks

prep_pack (per interview/company) | column | type | notes | |---|---|---| | id | bigint PK | user_id FK | | interview_id | bigint FK | | | topics | jsonb | [{topicSlug, why}] = JD topics ∩ weak ∩ gaps | | summary | text | AI-generated focus brief | | generated_at | timestamptz | |

star_story | column | type | notes | |---|---|---| | id | bigint PK | user_id FK | | title | varchar(200) | | | situation, task, action, result | text | | | tags | jsonb | themes: conflict, scale, failure… | | mapped_prompts | jsonb | likely behavioral prompts this answers |

mock_session | column | type | notes | |---|---|---| | id | bigint PK | user_id FK | | type | varchar(24) | `technical` | `system-design` | `behavioral` | | topic_scope | jsonb | topics covered (often weak ones) | | transcript | jsonb | turns | | score | int | 0–100 | | feedback | text | AI debrief | | created_at | timestamptz | |

> Mock results feed the loop: low-scored areas create `review_flag` s like interviews do.

4.7 Jobs, AI cache, agent audit

job_alert — `id, user_id, source, company, role, location, comp, url, jd_text, email_msg_id (unique), received_at, fit_score, fit_reason, status(new|interested|dismissed)`

ai_cache — `id, user_id, cache_key (unique), kind(guide|mock|analysis|jobfit|parse|plan), content, model, created_at`

agent_run — `id, user_id, agent, trigger_ref, steps_used, tokens_used, status(ok|budget_exceeded|timeout|error), result_summary, created_at`

4.8 Relationship summary

```
app_user 1—1 user_settings, 1—1 resume_profile 1—N resume_chunk
app_user 1—N phase 1—N week 1—N topic ─┬─1:1 note 1—N attachment
                                       ├─1:N resource | exercise | question | review_flag
interview 1—N interview_question N—1 topic ;  interview 1—1 prep_pack
app_user 1—N star_story | mock_session | job_alert | usage_log | agent_run
ai_cache, resume_chunk : standalone (vector/cache)
```

---

5. Java core — REST API

Base `/api`. JWT required except `/api/auth/**`. Each line: **what / why**.

5.1 Auth & onboarding

```
POST /api/auth/login                 issue JWT / why: remote access, single user
POST /api/auth/refresh               rotate token
GET  /api/me                         current user + settings
POST /api/onboarding/resume          upload resume (multipart) → calls AI parse → returns
                                     draft resume_profile / why: start onboarding
PUT  /api/onboarding/profile         save user-edited profile (confirm step) / why: parsing is lossy
POST /api/onboarding/plan            inputs {targetRole, level, weeks, hoursPerWeek,
                                     interests[], budget, llm} → calls AI plan-gen →
                                     returns draft plan (phases/weeks/topics) / why: tailored plan
PUT  /api/onboarding/plan/commit     persist edited plan, finish onboarding
```

5.2 Study plan & topics

```
GET   /api/phases                    full plan tree (light topics)
GET   /api/topics/{slug}             full topic detail (concept, note, resources, exercises, questions)
POST  /api/topics                    add custom topic
PATCH /api/topics/{slug}             edit / mark done / set confidence
DELETE/api/topics/{slug}             remove custom topic
GET   /api/progress                  overall %, per-track readiness, streak, flag counts
```

5.3 Notes / resources / exercises / questions

```
GET  /api/topics/{slug}/note         markdown + attachments
PUT  /api/topics/{slug}/note         upsert note (Notion-style editor saves here)
POST /api/topics/{slug}/note/attachments   (multipart) image upload
GET/DELETE /api/attachments/{id}     serve / delete file
CRUD /api/topics/{slug}/resources    label+url links
CRUD /api/topics/{slug}/exercises    title+repoUrl+done (GitHub link, Monaco view)
CRUD /api/topics/{slug}/questions    study Q-bank
```

5.4 Interviews, prep packs, STAR, mocks

```
GET  /api/interviews                 pipeline board (grouped by stage)
GET  /api/interviews/{id}            debrief detail (+ questions, weak[])
POST /api/interviews                 add (company, role, stage, date, jd_text?)
PATCH/api/interviews/{id}            progress stage/outcome; attach jd_text
POST /api/interviews/{id}/questions  log asked question {text, topicSlug?, selfRating}
                                     / why: THE feedback loop (≤2 → flag + lower confidence)
POST /api/interviews/{id}/prep-pack  generate company pack from jd_text ∩ weak ∩ gaps
GET  /api/interviews/{id}/prep-pack  fetch pack
CRUD /api/star-stories               behavioral story bank
GET  /api/mocks  POST /api/mocks/start  GET /api/mocks/{id}   mock sessions (run via AI svc)
```

5.5 Review (adaptive) & jobs & usage

```
GET  /api/review/today               blended focus: new + flagged + drill (adaptive)
POST /api/review/{slug}/done         resolve flags, bump confidence, stamp reviewed
POST /api/plan/rebalance             trigger coach-agent re-plan from progress+flags / why: adaptive
GET  /api/jobs?sort=fit&status=      scored job feed (from Gmail ingestion)
PATCH/api/jobs/{id}                  interested/dismissed
GET  /api/usage                      spend this month + remaining (budget meter)
```

5.6 AI gateway (proxied to Python; metered)

```
GET  /api/ai/guide/{slug}            deep-dive guide (cached)
POST /api/ai/mock/{slug|type}        run/continue a mock
POST /api/ai/analyze                 weekly interview-pattern analysis
```

Java forwards these to the Python service, applying `BudgetGuard` and writing `usage_log`.

---

6. Python AI service — FastAPI surface

Internal, called by Java (service token). Each returns `{result, meta:{model,cached,tokens,cost}}`.

```
POST /ai/parse-resume        {rawText}              → structured profile + gaps
POST /ai/generate-plan       {profile, targets}     → phases/weeks/topics (incl DSA level, interests)
POST /ai/embed-resume        {experiences[]}        → write resume_chunk vectors (one-time)
POST /ai/score-job           {jd}                   → {fit, reason}  (batched, cheap tier, RAG)
POST /ai/guide               {topic}                → study guide (cached)
POST /ai/debrief             {interviewId}          → tag questions, flag weak (agent)
POST /ai/prep-pack           {interviewId, jd}      → focus topics + brief (agent)
POST /ai/coach               {progress, flags}      → adaptive plan adjustments (agent)
POST /ai/mock/start|turn     {type, scope}          → mock interviewer turns + final score
POST /ai/analyze             {interviews}           → strategic patterns (weekly)
```

Provider abstraction: one `LLMProvider` interface (`complete`, `embed`, tool-calling) with implementations for Anthropic (default), Bedrock, OpenAI, local. Model ids + tier come from `user_settings`; keys from Vault. Cheap vs strong tier selected per task.

Agents (LangGraph, bounded ReAct): job-fit, debrief, coach, mock, prep-pack. Guards: max steps (3–6), 30s deadline, token budget, allow-listed tools, idempotent writes, full `agent_run` trace. **Agent tools = HTTP calls back to Java REST** (`search_topics`, `get_progress`, `flag_for_review`, `schedule_review`, `get_interview_log`) + local `vector_search` (pgvector). Same tools exported via **MCP server** for Claude Desktop.

---

7. Integrations

7.1 Gmail (job feed)

Read-only OAuth2 (`gmail.readonly`), refresh token in Vault. Scheduled poll (Java, `@Scheduled`) queries job-alert senders, parses postings (one parser per source, start LinkedIn), dedupes on `email_msg_id`, inserts `job_alert(status=new)`. Java then asks the AI service to batch-score new rows (cheap tier, RAG over `resume_chunk`).

7.2 LLM provider

Selected at onboarding, stored in `user_settings`, keys in Vault. Swappable without code change. Anthropic default; cheap-tier fallback enforced by `BudgetGuard`.

---

8. Adaptive plan logic

- `review/today` blends: next unstarted topics (plan order) + open interview flags (interview-specific stay scoped until that interview's date passes, then fold into the general pool) + a drill from a topic with exercises.
- `POST /plan/rebalance` (and a weekly cron) runs the **coach agent**: reads progress + open flags + upcoming interviews, reorders/insert-reviews, writes changes. The plan is never static — it tracks reality.
- Spaced repetition: `confidence` decays with time since `last_reviewed_at`; decayed topics resurface in `review/today` and pull down the readiness score.

---

9. Readiness score (deterministic, no AI)

Per-track and overall, computed in Java: `readiness = 100 * (done/total) * (1 − 0.4*openFlags/total) * recencyFactor`, clamped 0–100, where `recencyFactor` falls as topic confidences decay. Drives "are you ready / what's soft."

---

10. Build order (each slice runnable)

1. **Foundation** — docker-compose (Postgres, Vault), `app_user` + auth (JWT), Vault wiring, config, error model.
2. **Study core** — phase/week/topic + note/resource/exercise/question CRUD, Flyway, `/api/phases`, `/api/topics/**`, `/api/progress`, readiness score.
3. **Attachments** — FileStore + image upload for notes.
4. **Interviews + loop** — interview/question/review_flag, flag-on-rating rule, `/api/interviews/**`, `/api/review/today`.
5. **Python AI service skeleton** — FastAPI, provider abstraction, Vault keys, `usage_log`, BudgetGuard, `/ai/guide` (cached) wired through Java `/api/ai/guide`.
6. **Onboarding** — resume upload → `/ai/parse-resume` → confirm → `/ai/generate-plan` → commit. Embed resume chunks.
7. **Agents** — job-fit (+Gmail ingestion), debrief, coach (adaptive), prep-pack, mock interviewer; MCP server; `agent_run` audit.
8. **STAR bank, jobs feed UI data, usage meter** — finish remaining endpoints.

Wire the React UI to slice 2 immediately; build subsequent slices behind it.

---

11. Endpoint quick reference

```
AUTH/ONBOARD  /api/auth/login|refresh  /api/me
              /api/onboarding/resume|profile|plan|plan/commit
STUDY         /api/phases  /api/topics/{slug}[POST,PATCH,DELETE]  /api/progress
CONTENT       /api/topics/{slug}/note[GET,PUT]  /note/attachments  /api/attachments/{id}
              /api/topics/{slug}/resources|exercises|questions (CRUD)
INTERVIEWS    /api/interviews[GET,POST]  /{id}[GET,PATCH]  /{id}/questions
              /{id}/prep-pack[GET,POST]  /api/star-stories(CRUD)
              /api/mocks  /api/mocks/start  /api/mocks/{id}
REVIEW/JOBS   /api/review/today  /api/review/{slug}/done  /api/plan/rebalance
              /api/jobs  /api/jobs/{id}  /api/usage
AI GATEWAY    /api/ai/guide/{slug}  /api/ai/mock/{slug|type}  /api/ai/analyze
PYTHON (int)  /ai/parse-resume|generate-plan|embed-resume|score-job|guide|debrief
              /ai/prep-pack|coach|mock/start|mock/turn|analyze
```